

HUDSON DW v0 - Guia de comandos e funcionamento

Sugoi S.A. - Biblioteca soberana (HUDSON DW). Documento de referencia para o desenvolvimento do frontend. Baseado no codigo do repositório Sugoi-SA/HUDSON (pasta backend/, 552 linhas em app/ e scripts/), e nao na documentacao narrativa, que pode estar desatualizada.

1. Visao geral da arquitetura

O HUDSON DW v0 tem tres partes: (a) uma API HTTP em FastAPI, na porta 8000; (b) um CLI de importacao, que varre um diretorio de origem; (c) um banco PostgreSQL, com o schema **hudson**, e um diretorio de storage onde ficam as copias dos arquivos custodiados.

Componente	Arquivo	Funcao
API HTTP	app/main.py + app/routers/	Consulta por cota, busca full-text e health check
Importacao (CLI)	scripts/import_cli.py + app/services/importer.py	Varre diretorio, deduplica, roteia, copia, extrai texto e registra custodia
Seguranca	app/security.py	Autenticacao por chave unica no header X-API-Key
Banco	app/db.py, app/models.py, specs/S2-schema.sql	SQLAlchemy 2 sobre PostgreSQL, schema hudson
Storage	STORAGE_ROOT (variavel de ambiente)	Copia dos arquivos, organizada por estante e prefixo do hash

Configuracao (arquivo .env, lido por app/config.py): DATABASE_URL, STORAGE_ROOT, API_KEY (obrigatorias); TESSERACT_CMD e POPPLER_PATH (opcionais, para OCR no Windows). Sem essas variaveis a aplicacao nao inicia.

2. Interface HTTP (API)

A API tem apenas tres rotas. Nao existe POST /items, /authenticity, RBAC, relatorios ou exportacao no codigo atual, apesar de estarem descritos na especificacao S1 e no roadmap. Esse ponto deve ser considerado no desenho do frontend.

Metodo e rota	Autenticacao	Parametros	Resposta	Efeito na custodia
GET /health	Nenhuma	-	200 {"status": "ok"}	Nenhum
GET /items/{cota}	X-API-Key	cota (path)	200 ItemOut; 404 'Item nao encontrado'; 401 se chave invalida	Grava 'consulta' com item_id, hash e reason lookup_cota:<cota>
GET /search	X-API-Key	q (obrigatorio, min. 2 caracteres); limit (padrao 50, max. 200)	200 SearchResponse {query, total, results[]}; 401 se chave invalida	Grava 'consulta' com item_id NULL, hash 'n/a' e reason busca:<q>

Campos de ItemOut: cota, hash_sha256, estante, status, storage_path, obra_wbs, received_at, processed_at. O campo **id (UUID)** **nao e retornado**, o que limita a navegacao entre itens e custodia pelo frontend.

Campos de SearchResultItem: cota, estante, status, storage_path, received_at, rank (float, relevancia ts_rank). A busca retorna somente itens com texto indexado (search_vector preenchido).

Exemplos (substituir CHAVE pela API_KEY e o host pelo IP do servidor):

```
curl http://192.168.1.122:8000/health
```

```
curl -H "X-API-Key: CHAVE" "http://192.168.1.122:8000/items/COTA_AQUI"
```

```
curl -H "X-API-Key: CHAVE" "http://192.168.1.122:8000/search?q=contrato%20obra&limit=20"
```

Documentacao interativa: o FastAPI gera automaticamente /docs (Swagger) e /openapi.json. Verificar se estao expostos no ambiente de producao antes de divulga-los.

3. Importacao por linha de comando (CLI)

Comando para varrer um diretorio, recursivamente, e custodiar os arquivos. O WBS define a parte da cota do item.

```
cd backend
.venv/bin/python -m scripts.import_cli --source /caminho/da/origem --wbs OBRA01
```

Parametros: **--source** (obrigatorio, diretorio a varrer); **--wbs** (padrao INDEFINIDO, codigo do empreendimento).
Saida: total varrido, novos itens, indexados (com texto), sem texto extraido, sem estante mapeada, duplicados e lista de erros.

Fluxo por arquivo (cada arquivo tem commit proprio; uma falha em um arquivo nao interrompe os demais):

Etapa	O que acontece	Evento de custodia
1. Hash	SHA-256 em streaming, blocos de 1 MiB	-
2. Deduplicacao	Se o hash ja existe, nao cria item novo	deduplicacao
3. Criacao	Item criado com status 'recebido'	recebimento
4. Roteamento	Estante definida pela extensao. Sem mapeamento: alerta e o arquivo NAO e copiado	roteamento ou alerta_integridade
5. Cota	Gerada, se houver estante (formato na secao 4)	-
6. Copia	Copia para STORAGE_ROOT/estante/hash[0:2]/hash.ext, com verificacao do hash. Falha: status falha_processamento	alerta_integridade (se falhar)
7. Extracao	Texto por tipo (secao 4). Com texto: indexa tsvector em portgues	indexacao ou processamento
8. Fechamento	Status final e processed_at	processamento (sem texto)

4. Organizacao dos arquivos

Estantes (mapeadas pela extensao, em app/services/estante_router.py):

Estante	Extensoes	Extracao de texto
document_text	.pdf, .doc, .docx, .odt, .rtf, .txt	PDF: pypdf; se o texto tiver menos de 20 caracteres, OCR com pdf2image + Tesseract (idioma por). .docx: python-docx. .txt: leitura direta. .doc, .odt e .rtf: sem texto
communication	.eml, .msg, .pst	Sem extracao (indexacao nao implementada)
engineering_drawing_s	.dwg, .dxf, .rvt, .ifc	Sem extracao
structured_data	.xls, .xlsx, .csv, .xml, .json	.csv: leitura direta. Demais: sem extracao
image	.jpg, .jpeg, .png, .tif, .tiff, .bmp, .gif	OCR com Tesseract (idioma por)
audio_video	.mp3, .wav, .mp4, .avi, .mov, .mkv	Sem extracao

Extensao fora da lista: o item e criado com status 'recebido', o arquivo nao e copiado, storage_path fica vazio, nao recebe cota e nao e encontrado por GET /items. Esse comportamento precisa ser tratado no frontend (lista de pendencias).

Layout do storage: STORAGE_ROOT/<estante>/<hash[0:2]>/<hash>.<ext>. O nome do arquivo e o proprio hash, o que garante que o mesmo conteudo nunca e gravado duas vezes.

Cota HUDSON (app/services/cota.py): *ESTANTE-WBS-ANO-HASH*, por exemplo *document_text-OBRA01-2026-3fa2b9c1*. O hash comeca com 8 caracteres e, se houver colisao, aumenta de 2 em 2 ate 64. Gerada com o ano atual (UTC), na data da importacao.

Status do item:

Status	Quando	Proximo
recebido	Item criado, antes do roteamento	roteado (ou permanece se nao houver estante)
roteado	Estante e cota definidas	processando
processando	Copia em storage verificada, extracao em andamento	indexado ou processado
indexado	Texto extraido e indexado para busca	-
processado	Processado sem texto extraido	-
falha_processamento	Hash da copia nao confere com a origem	-

Observacao: o schema e os diagramas citam estados adicionais (como 'retido', 'liberado' e 'reprocessamento'), mas nao ha codigo que os produza nesta versao.

5. Custodia (custody_log)

Tabela append-only, particionada por ano (2025, 2026, 2027). Cada linha registra um evento de custodia. Triggers bloqueiam UPDATE e DELETE em custody_log e DELETE em items. Os 12 tipos de evento validos sao:

Tipo de evento	Gerado pelo codigo hoje?	Quem gera
recebimento	Sim	Importador
deduplicacao	Sim	Importador (arquivo repetido ou concorrencia)
roteamento	Sim	Importador
alerta_integridade	Sim	Importador (extensao nao mapeada ou copia corrompida)
processamento	Sim	Importador (sem texto ou falha na extracao)
indexacao	Sim	Importador (texto indexado)
consulta	Sim	API (GET /items e GET /search)
retencao, liberacao, falha_llm, reprocessamento, lock_custodia	Nao	Previstos no CHECK do schema, sem codigo

O campo **actor** identifica quem gerou o evento: importer_v0 (CLI) ou api_v0 (API). O campo **payload_hash** guarda o SHA-256 do arquivo, ou 'n/a' nas buscas.

6. Acesso ao banco (SQL)

Schema: **hudson**. Tabelas: items, custody_log (com particoes custody_log_2025, _2026, _2027), entities, relationships, declarations, estant_types. Para o frontend, o acesso deve ser feito pela API, e nao diretamente pelo banco.

Colunas principais de items: id (uuid), hash_sha256 (unico), storage_path, status, estante (FK para estant_types.codigo), cota (unica), obra_wbs, duplicate_of_item_id (FK para items.id), received_at, processed_at, search_vector (tsvector).

Colunas de custody_log: id (bigint), item_id (FK, pode ser nulo), event_type, actor, reason, timestamp (chave de particao), payload_hash.

Consultas de leitura (executar no container: docker exec -it sugoi-postgres psql -U postgres -d sugoi -P pager=off -c "..."):

```
-- Itens por estante e status
SELECT estante, status, count(*) FROM hudson.items GROUP BY 1, 2 ORDER BY 1, 2;

-- Ultimos 20 eventos de custodia
SELECT timestamp, event_type, actor, reason FROM hudson.custody_log ORDER BY timestamp DESC LIMIT 20;

-- Historico de custodia de uma cota
SELECT c.timestamp, c.event_type, c.actor, c.reason
FROM hudson.custody_log c JOIN hudson.items i ON i.id = c.item_id
```

```
WHERE i.cota = 'COTA_AQUI' ORDER BY c.timestamp;
```

```
-- Arquivos que precisam de atencao (sem estante ou com falha)
SELECT hash_sha256, status, storage_path FROM hudson.items
WHERE status IN ('recebido', 'falha_processamento');
```

Escrita manual: nao recomendada. Qualquer INSERT deve seguir as regras de custodia (evento correspondente). UPDATE e DELETE em custody_log e DELETE em items sao bloqueados pelo banco.

7. Pesquisas

A busca usa full-text search em portugues: plainto_tsquery('portuguese', q) para a consulta e ts_rank para ordenar. Consulta equivalente em SQL:

```
SELECT i.cota, i.estante, i.status, i.storage_path,
       ts_rank(i.search_vector, plainto_tsquery('portuguese', 'contrato obra')) AS rank
FROM hudson.items i
WHERE i.search_vector @@ plainto_tsquery('portuguese', 'contrato obra')
ORDER BY rank DESC LIMIT 50;
```

Limitacoes: a busca cobre apenas os itens com texto extraido. Imagens dependem da qualidade do OCR. Nao ha filtros por estante, WBS ou periodo na API, o que precisa ser implementado para o frontend.

8. Relatorios

Nao ha endpoint nem comando de relatorio na versao atual. Os dados podem ser extraidos com SQL (secao 6). Relatorios recorrentes devem ser definidos como novas rotas da API, para manter a regra de custodia em um unico lugar.

```
-- Volume por estante (para relatorio mensal)
SELECT estante, count(*) AS itens, min(received_at) AS primeiro, max(received_at) AS ultimo
FROM hudson.items WHERE estante IS NOT NULL GROUP BY 1 ORDER BY 1;
```

9. Backup e restore

O servidor atual usa as ferramentas do container sugoi-postgres (PostgreSQL 18.6). O backup manual e feito com pg_dump e pode ser verificado com pg_restore --list, sem restaurar. A validacao completa (restore em banco temporario, triggers e teste de imutabilidade) foi feita em 2026-10-07.

```
mkdir -p /home/hudson/backups/docker
docker exec sugoi-postgres pg_dump -U postgres -Fc -d sugoi > /home/hudson/backups/docker/sugoi_$(date +%Y%m%d_%H%M%S).dump
docker exec -i sugoi-postgres pg_restore --list < /home/hudson/backups/docker/ARQUIVO.dump | head -40
```

Ressalva: o agendamento automatico do script antigo (deploy/backup.sh) aponta para o banco hudson do ambiente v0, que nao existe no container atual. Nao ha backup automatico do banco sugoi confirmado.

10. Limitacoes e pontos de atencao para o frontend

Lista do que o frontend precisa considerar, com base no codigo:

Ponto	Situacao atual
Autenticacao	Chave unica (X-API-Key). Nao ha usuarios, perfis nem sessao. RBAC esta na Fase 1.1.
Escrita pela API	Nao existe. Itens so entram pelo CLI. O frontend nao pode enviar arquivos.
Identificador	ItemOut nao retorna o id (UUID). A consulta e feita por cota.
Cota ausente	Itens sem estante nao tem cota e nao sao consultaveis pela API.
Busca	Somente texto indexado. Sem filtros nem paginacao alem de limit (max. 200).
Download	Nao existe rota para baixar o arquivo. storage_path e um caminho do servidor.
Historico	Nao existe rota para a custodia. Consulta somente via SQL.
Protecao	Trafego HTTP em texto claro, na porta 8000. Recomendado HTTPS interno.

Ponto	Situacao atual
Erros	Respostas de erro padrao do FastAPI ({'detail': ...}). Sem codigo de erro proprio.

Rotas a criar para o frontend (sugestao, a decidir): listagem paginada de itens com filtros; detalhe por id; historico de custodia; download com verificacao de hash; upload com a mesma regra do CLI; relatorios; autenticacao por usuario.